Diagnose Before Solving: AI Simulation

Embedding and LMS Readiness Note (v1.3)

Project owner: David Bacon
Document status: Revised for publication. Retained as the embedding contract; see the status box below.
Date: August 31, 2026 (v1.0); status update November 11, 2026 (v1.1); revised November 12, 2026 (v1.2 and v1.3)
Deliverable: 5 of 11
Depends on: Scope Document v1.2, Technical Design v1.3

> **Revision note (v1.2, November 12, 2026).** Retitled from "Storyline Integration and LMS Readiness Note", since Storyline is now one possible host among others (D-188); the filename is unchanged so existing references keep working (D-186). The status box is brought up to date: the embed capability was kept (D-188), the other documents were revised to match, and the capability has been verified against a live test host (D-192, D-202). The first troubleshooting row in section 7 is corrected: an absent `frame-ancestors` directive does not blank a frame, it permits framing (D-200, D-203). Section 9 questions 1 and 4 are resolved. The payload, the listener code and the LMS mapping are unchanged.

> **Revision note (v1.3, November 12, 2026).** Section 9 question 2 is resolved: the simulation now posts to a named target origin, `https://dbacon6093.github.io`, rather than `*` (D-219). Until then the code posted to `*` while Technical Design section 11 said otherwise. A host on another origin now receives nothing unless it is added to the simulation's configuration, so the checklist in section 6 and the troubleshooting table in section 7 each gain a line. The payload, the listener code and the LMS mapping are unchanged.

Status after November 12, 2026 (D-173, D-177, D-188)

What changed. The training that precedes the simulation is no longer a Storyline module. It is a separate static site hosted on GitHub Pages, and it reaches the simulation through an ordinary link (D-173, D-177). Nothing is passed between the two sites.

What this document is now. Sections 2 through 8 describe the contract between the simulation and any page that embeds it in a frame. Storyline is the worked example because it was the host this was designed for. **No host page in the learner-facing deployment uses it:** learners reach the simulation by link. Read sections 2 to 7 as a design record and as the instructions to follow if the simulation is ever embedded in a Storyline module or an LMS-hosted page.

The embed capability was kept, and it is verified against a test host. David chose to keep embedding as a verified feature rather than remove it (D-188). A plain test page, the embed harness, frames the production simulation from a different origin over HTTPS and checks every completion message against the payload specification in section 3 (Test Plan section 9.1). On production it rendered the simulation in a frame, received one complete message per completion, and passed all 13 of its payload checks (D-192, D-193). The simulation now sends a restrictive `frame-ancestors` policy that permits the harness's origin, and a framed render under that policy was confirmed live (D-201, D-202).

What that verification does not cover. It is not a Storyline or LMS result. Storyline's player and LMS wrappers add iframe layers, messages and restrictions the harness does not reproduce (Test Plan section 17). The honest claim is **embeddable and LMS-mappable by design, verified with a test host**.

Where the rest of the project describes this capability. Scope 6.3 and success criterion 6, Technical Design sections 10 and 11, and Test Plan tests F3 to F11 and F20 to F22 were all revised to treat Storyline as one possible host and to verify against the harness (D-188). Decision Log Part 1 (D-016, D-023, D-027, D-028, D-085, D-086, D-088) is frozen and still records the Storyline plan as it was decided.

1. What this document is

The simulation and any page that hosts it are separately owned. This document is the contract between them: what the simulation sends when a learner finishes, how a host receives it, and what a person would need to do to connect it to an LMS.

It has two audiences.

The person building a host page needs the setup steps, the code to paste, and the troubleshooting guide. Sections 3 through 7, written for a Storyline module.

A portfolio reviewer is looking for evidence that the designer understands how a custom activity connects to a learning ecosystem. Sections 2 and 8 carry that, and the LMS mapping in section 8 is the specific thing a technically literate reviewer will look for.

Scope boundary. The simulation sends the message. A host page receives it. This project builds the first half and specifies the second; any host is built separately (D-016). As of D-173 no Storyline module is planned for the training; the specification stands for any host that embeds the simulation.

---

2. Why this is not as simple as it looks

The obvious way to set a Storyline variable from inside a web object is to call Storyline's own player object directly:

```
        parent.GetPlayer().SetVar("simScore", 85);   // Does not work here
```

This works only when the web object and the Storyline module are served from the same origin: the same protocol, domain, and port. Here they will not be. The simulation runs on Vercel; the module is published somewhere else entirely. The browser blocks a script in one origin from reaching into another, and it does so **silently**, which is how an afternoon disappears.

The supported path across origins is `postMessage`, a browser mechanism specifically designed for two pages on different origins to exchange data safely. It works in two halves:

1. The simulation calls `window.parent.postMessage(payload, targetOrigin)` when the learner finishes.
2. The module registers a listener that receives the message, checks who sent it, and calls `SetVar()` from there.

The module side is a single JavaScript trigger. It is not difficult, but it will not happen by accident, and nothing about the failure tells you what went wrong.

---

3. The payload

Sent once, when the simulation reaches a terminal state and scoring is complete.

```
{
  "source": "diagnose-before-solving",
  "version": 1,
  "event": "simulation_complete",

  "score": 38,
  "maxScore": 100,
  "passed": false,
  "passThreshold": 70,

  "completionStatus": "completed",
  "outcome": "turn_limit",
  "turnsUsed": 5,
  "escalationCount": 2,

  "criteria": [
    { "id": 1, "level": "partial", "points": 13, "max": 25 },
    { "id": 2, "level": "full",    "points": 25, "max": 25 },
    { "id": 3, "level": "zero",    "points": 0,  "max": 20 },
    { "id": 4, "level": "partial", "points": 10, "max": 20 },
    { "id": 5, "level": "partial", "points": 5,  "max": 10 }
  ],
  "deductions": [
    { "id": "D1", "applied": false, "points": 0   },
    { "id": "D2", "applied": true,  "points": -15 },
    { "id": "D3", "applied": false, "points": 0   }
  ]
}
```

Field	Values
`completionStatus`	`completed`, `abandoned`
`outcome`	`resolved`, `escalated`, `off_topic`, `turn_limit`
`criteria[].level`	`full`, `partial`, `zero`

The narrative feedback is deliberately not in the payload (D-027). It is long, quotes specific turns, and is formatted; pushing it into Storyline text boxes means fixed-size containers, no scrolling, and lost formatting. The learner reads it in the simulation's own results view, which is built for it.

`source` and `version` exist because your listener will receive other messages. The Storyline player posts its own messages, browser extensions post messages, and embedded players post messages. A listener that acts on everything it receives will break in ways that are hard to reproduce. Always check `source` first.

---

4. Storyline setup

Step 1: Create the variables

In the Variables panel, before anything else. Names must match the code exactly, including capitalisation.

Variable	Type	Initial value
`simComplete`	True/False	False
`simAttempt`	Number	0
`simScore`	Number	0
`simPassed`	True/False	False
`simOutcome`	Text	(empty)
`simEscalations`	Number	0

Optionally, for per-criterion display or remediation branching: `simC1` through `simC5`, all Number.

`simAttempt` exists because retries are permitted (D-070), and it solves a problem that would otherwise be invisible. Storyline's "when the variable changes" trigger fires only when a variable's value actually differs from what it held before. After a first attempt, `simComplete` is already True; setting it to True again on a retry **is not a change**, so

a results trigger watching `simComplete` would fire on the first attempt and never again. The learner would retry, score differently, and see their old result.

`simAttempt` increments on every completion, so its value always differs and the trigger always fires. The two variables then have distinct jobs:

Variable	Job	Used as
`simComplete`	Has the learner finished the activity at least once? Stays True once earned	A **condition**, for gating navigation
`simAttempt`	A completion just happened	A **change trigger**, for showing or refreshing results

Step 2: Insert the web object

Insert → Web Object, then:

- **Address:** `https://diagnose-before-solving.vercel.app/?embed=1` The parameter suppresses the landing view, since the module has already set the scenario (Scope 4.1, D-055).

  Use the production domain, not a deployment URL. Vercel issues every individual deployment its own address containing a random segment, of the form `project-a1b2c3d4-team.vercel.app`. That address is valid but temporary: it changes on the next push. A module built against one will work when tested and break silently the next time the application is deployed, which is a painful failure to diagnose weeks later. The production domain is the one without the random segment.

- **Display in:** *Slide*. Not "new browser window". Displaying in the slide is what creates the iframe this whole mechanism depends on.
- Size it generously. See the note on story size in section 7.

Step 3: Add the listener

Add a trigger on the slide holding the web object:

> **Execute JavaScript** when the **timeline starts** on this slide

Paste this as written. `APP_ORIGIN` is already set to the production domain, with no trailing slash:

```
(function () {
  // Guard against attaching twice if the learner revisits this slide.
  if (window.__dbsListenerAttached) { return; }
  window.__dbsListenerAttached = true;

  var APP_ORIGIN = "https://diagnose-before-solving.vercel.app";

  window.addEventListener("message", function (event) {
    // 1. Only accept messages from the simulation's origin.
    if (event.origin !== APP_ORIGIN) { return; }

    // 2. Only accept messages that are actually ours.
    var d = event.data;
    if (!d || d.source !== "diagnose-before-solving") { return; }
    if (d.event !== "simulation_complete") { return; }

    // 3. Write the values into Storyline.
    var player = GetPlayer();
    player.SetVar("simScore", d.score);
    player.SetVar("simPassed", d.passed);
    player.SetVar("simOutcome", d.outcome);
    player.SetVar("simEscalations", d.escalationCount);

    // Optional per-criterion values
    if (d.criteria) {
      for (var i = 0; i < d.criteria.length; i++) {
        player.SetVar("simC" + d.criteria[i].id, d.criteria[i].points);
      }
    }

    // 4. Flags LAST, and simAttempt very last. See the notes below.
    player.SetVar("simComplete", true);
    player.SetVar("simAttempt", (player.GetVar("simAttempt") || 0) + 1);
  });
})();
```

Four details in that code are load-bearing.

The **re-entry guard** at the top matters because a learner who navigates away and returns would otherwise attach a second listener, and every subsequent message would be handled twice.

The **origin and source checks** are what make this safe. Without them the listener acts on any message any page sends it.

Setting the flags after the data is not stylistic. Storyline triggers fire the moment a variable changes. If a flag were set first, a trigger watching it would run while `simScore` was still zero, and the feedback slide would show a score of nothing. Set the data, then raise the flags.

`simAttempt` is incremented very last, and it is what triggers should watch. On a retry, `simComplete` is already True and setting it to True again is not a change, so a trigger watching it will not fire a second time. `simAttempt` always differs from its previous value, so it always fires.

Step 4: Act on the result

Watch `simAttempt`, not `simComplete`:

> **Jump to slide** [Results] when the **variable** `simAttempt` **changes**

No condition is needed, since `simAttempt` only ever changes on a completion. This fires on the first attempt and on every retry.

Use `simComplete` as a condition, not a trigger:

> **Disable** the Next button until `simComplete` = True

Then branch on `simPassed` for a pass or remediation path, and display `%simScore%` wherever the number should appear. Because each completion overwrites the previous values, the results slide always shows the most recent attempt (D-070).

If you ever want to show growth across attempts, capture the first attempt into a separate variable with a trigger conditioned on `simAttempt` = 1. Not needed now; noted because it is easier to add at build time than later.

---

5. Recommended module structure

Not required, and any module is outside this project's scope. Offered because the integration works better with some patterns than others. The teaching principles in this section still apply to the separate training site (the Lead-in Outline cites them), even though its mechanics differ.

Gate the Next button on completion. Disable navigation off the simulation slide until `simComplete` is True. Without this, a learner can click past the activity and the module records nothing. This is the single most valuable thing the completion flag buys you.

Put the simulation on its own slide. It needs the room, and a learner should not be reading other content while Marcus waits.

Teach before, do not remind during. The rubric is not shown in the simulation, by design (D-055). Whatever instruction precedes the activity is the only preparation the learner gets, which is how a real call works.

Debrief after, using the score rather than repeating the feedback. The simulation gives detailed narrative feedback in its results view. A Storyline debrief slide adds most value by doing what the simulation cannot: connecting this one call to the wider course, offering a retry, or routing to remediation on a low criterion 3 score.

Retries are permitted, and the most recent attempt is what counts (D-070). The simulation restarts cleanly and a second attempt after reading the feedback is where most of the learning happens. Each completion overwrites the previous values, so the module always holds the latest result.

This is a defensible choice to be able to explain: reporting the most recent attempt measures what the learner can do *now*, after remediation, rather than how they performed cold. A best-of-attempts rule would reward persistence, and a first-attempt rule would measure unprepared competence; most-recent measures current capability, which is what a support organisation actually cares about.

---

6. Testing checklist

In order. Most integration failures are caught by the first two lines.

- ☐ **Test in a published module, not in Storyline preview.** Preview runs from a different origin, sometimes a local file, where origin checks behave differently. A preview result, pass or fail, tells you very little.
- ☐ **Serve the published module over HTTPS.** A browser will refuse to load an insecure frame inside a secure page, and it will not be obvious that this is what happened.
- ☐ **Add the module's origin to the simulation's `ALLOWED_FRAME_ANCESTORS` and redeploy the simulation** before testing. The deployed policy permits only the simulation itself and `https://dbacon6093.github.io` (D-202); any other host gets a blank frame until it is added.
- ☐ **Also set `NEXT_PUBLIC_COMPLETION_TARGET_ORIGIN` to the module's origin and redeploy** (D-219). The simulation posts its completion message only to `https://dbacon6093.github.io` unless told otherwise. A host on any other origin will frame the simulation correctly and never receive the score.
- ☐ Web object renders the simulation rather than a blank rectangle.
- ☐ Open the browser console and confirm there are no Content Security Policy or frame errors.
- ☐ Complete a simulation and confirm the variables populate. The quickest check is a temporary text box on the slide reading `%simScore%` and `%simComplete%`.
- ☐ Confirm the completion trigger fires and branching works for both a pass and a fail.

- [ ] **Run a retry and confirm the results slide updates.** This is the specific case that fails if a trigger watches `simComplete` instead of `simAttempt`, and it fails by showing the learner their previous score, which is easy to miss if the two attempts happen to score similarly. Deliberately score very differently on the two attempts when testing this.
- [ ] Revisit the slide and run a second simulation, confirming the listener guard holds and values update correctly.
- [ ] If the module will be hosted in an LMS, repeat the whole checklist there. The LMS is another iframe layer and another origin.

---

7. Troubleshooting

Symptom	Likely cause
Web object is a blank rectangle	Framing headers. Either the simulation's `frame-ancestors` directive does not list this host's origin, or an `X-Frame-Options` header is present. The simulation must never send `X-Frame-Options`. **An absent `frame-ancestors` directive is not the cause:** it permits framing from anywhere, which production showed for six weeks before the header was built (D-200). Since D-202 the directive lists only `'self'` and `https://dbacon6093.github.io`, so a new host must be added to `ALLOWED_FRAME_ANCESTORS` and the simulation redeployed. The project's likeliest silent failure (Technical Design section 10)
Blank rectangle, module served over HTTPS	Mixed content: an `http://` web object address inside an `https://` page. Use `https://`
Simulation loads, message never arrives	`APP_ORIGIN` does not exactly match the simulation's origin. It must include the protocol and no trailing slash: `https://app.vercel.app`, not `app.vercel.app` or `https://app.vercel.app/`
Worked at first, then stopped after a deploy	A **per-deployment URL** was used instead of the production domain. Vercel gives every deployment its own address containing a random segment (`project-a1b2c3d4-team.vercel.app`), and that address changes on every push. The web object address and `APP_ORIGIN` must both use the stable production domain
Frame renders, message never arrives, no error anywhere	The simulation's target origin does not match this host. A browser drops a `postMessage` whose target origin differs from the receiving page's origin, silently. Set `NEXT_PUBLIC_COMPLETION_TARGET_ORIGIN` to the host's origin (no trailing slash) and redeploy (D-219). Check this before debugging the listener
Message never arrives, origin looks right	The listener never attached. Confirm the trigger is on the slide holding the web object and fires on timeline start
Variables stay at their initial values	Name or type mismatch. `simScore` must exist as a Number and be spelled identically in both places
Feedback slide shows a score of zero	`simComplete` was set before the other variables. It must be set last (section 4, step 3)
Everything fires twice	The re-entry guard was removed, or the trigger is on a slide the learner revisits
Retry shows the previous attempt's score	A trigger is watching `simComplete`, which no longer changes once it is True. Watch `simAttempt` instead (section 4, step 4)
Retry does not start a fresh	The simulation's restart control creates a new session server-side. If Marcus remembers the earlier call, the page was not reset

conversation	
Works in preview, fails when published	Almost always the origin check. Preview and published output are different origins
Works locally, fails in the LMS	The LMS adds another iframe layer. Check `frame-ancestors` includes the LMS domain

8. LMS readiness

There is **no LMS integration in the portfolio deployment** (D-028). This section documents how the completion payload maps to standard LMS data elements, so that connecting it is a small, obvious step rather than a redesign.

The mapping is the point. A claim that something is LMS-ready, backed by a field mapping, is credible. The same claim backed by an arbitrary payload is not.

Payload field	SCORM 1.2	SCORM 2004	xAPI
`score`	`cmi.core.score.raw`	`cmi.score.raw`	`result.score.raw`
`score` / `maxScore`	`cmi.core.score.max`	`cmi.score.scaled`	`result.score.scaled`
`passed`	`cmi.core.lesson_status` (`passed` / `failed`)	`cmi.success_status`	`result.success`
`completionStatus`	`cmi.core.lesson_status` (`completed` / `incomplete`)	`cmi.completion_status`	`result.completion`
`criteria[]`	(not supported)	`cmi.interactions.n`	`result.extensions`
`outcome`, `escalationCount`, `turnsUsed`	(not supported)	Suspend data or interactions	`result.extensions`

How it would actually be wired (revised November 11, 2026; D-173, D-187). The original version of this paragraph said that Storyline already reports to an LMS when published for SCORM or xAPI, so the received variables could drive a Storyline result and no custom LMS code would be required. That route depended on a Storyline module hosting the simulation. The training is no longer a Storyline module, and no learner-facing page hosts the simulation in a frame, so the route does not exist in the current deployment.

What is still true: the payload is shaped to map onto the elements in the table above, and nothing in the simulation needs to change for the mapping to hold. As specified in Technical Design section 11, the simulation posts its results to a parent window when it finishes; with no parent page listening, the message goes nowhere and does no harm. The embed harness shows the message arriving complete and well formed (D-192).

What it would take to report to an LMS today: a page that embeds the simulation, receives the `postMessage` payload, and writes it to the LMS. That page could be a Storyline module built as sections 4 and 5 describe, or a small SCORM or xAPI wrapper written for the purpose. Neither is built. The boundary in D-028 (no LMS code in the portfolio deployment) still holds, so the honest claim is **LMS-mappable by design, not LMS-connected**.

Per-criterion reporting is the more sophisticated half, and the reason `criteria[]` is in the payload even though completion and branching do not need it. SCORM 2004 interactions and xAPI result extensions can carry each criterion as its own reported outcome, which turns a single score into diagnostic data: an organisation could see that its support specialists reliably acknowledge frustration and reliably fail to diagnose before solving. That is the kind of reporting that makes a simulation worth deploying at scale, and the payload is shaped so it remains available.

What is deliberately not built: no SCORM, xAPI, or LTI code exists in the application. The mapping is documented; the wiring is not. That boundary is held on purpose (D-028) to prevent scope creep toward an integration nobody has asked for.

8a. Future direction: a live state channel (V2)

Recorded for future development, not built in V1. (This direction assumed a Storyline host that shows an avatar; it applies only if the simulation is embedded in one. See D-088.)

The idea. The simulation sends Marcus's emotional state to Storyline on every turn, not only at completion, so the module can show an avatar that changes with the conversation: visibly angry at the start, softening as the learner handles him well, hardening again if they mishandle him.

Most of this already exists. The performer emits `intensity` with every response (D-030), currently used for continuity, ending detection, and telemetry. V1 sends one `postMessage` at completion; V2 would add a lightweight message per turn carrying the current state. The `source` and `version` fields already in the payload (D-066) mean a new event type can be added without breaking a listener built against V1.

Shape of the addition:

```
{ "source": "diagnose-before-solving",
  "version": 2,
  "event": "state_update",
  "mood": "hostile",
  "turn": 4 }
```

On the Storyline side: a Text variable such as `simMood`, and a trigger showing the matching avatar layer when it changes.

The design question this raises, and why it is not a contradiction. D-025 and D-080 deliberately excluded any anger meter or emotional-state display from the simulation's own interface, on the grounds that it turns a conversation into a game to be optimized. An avatar looks like the same thing and is not, for a reason worth writing down before someone re-litigates it:

A meter adds a channel that a real call does not have. An avatar **restores** one that a real call does have and that text chat strips out. On a real support line the specialist hears the anger in someone's voice, and reading it is part of the skill. Removing every affective cue does not make the exercise more rigorous; it makes it less like the thing being practised. Voice interaction, the original stretch goal, would restore the same channel more completely, and an avatar is a cheaper approximation of it.

One constraint that keeps the distinction real: keep the states coarse. If the avatar has eight states mapping one-to-one onto L0 through L4 and S1 through S3, learners will reverse-engineer the ladder and it becomes a meter with a face. Three or four broad states (hostile, guarded, engaged, resolved) read as emotional tone rather than as a score, which is the point.

Two other things to preserve. The avatar changes at the moment Marcus's reply appears, never before it, or it telegraphs the response. And `intensity` stays out of the evaluator's inputs (D-049) regardless: the learner may see it, but the score must still be derived from what was actually said.

9. Open questions

1. ~~**The Storyline module's published host**, needed for the `frame-ancestors` setting in the simulation.~~ **Resolved.** No Storyline module exists (D-173). The setting is now driven by `ALLOWED_FRAME_ANCESTORS` and lists the embed harness's origin, `https://dbacon6093.github.io` (D-201, D-202). A future host is added there, followed by a redeploy.
2. ~~**Whether the simulation posts to a named target origin or to `*`.**~~ **Resolved: a named origin** (D-219). The code posts to `https://dbacon6093.github.io`, the same origin the framing allowlist permits, and a different host is added through `NEXT_PUBLIC_COMPLETION_TARGET_ORIGIN`. One value is supported. The earlier code posted to `*`, which the payload's lack of sensitive data made tolerable but which did not match Technical Design section 11.
3. ~~**Whether retries are permitted, and whether a retry overwrites the previous score.**~~ **Resolved: retries are permitted and the most recent attempt is the one reported.** Each completion overwrites the previous values. See D-070 and sections 4 and 5.
4. ~~**Whether the simulation's embed capability stays in scope as a verified feature.**~~ **Resolved: it stays,** verified with a test host rather than in Storyline (D-188).